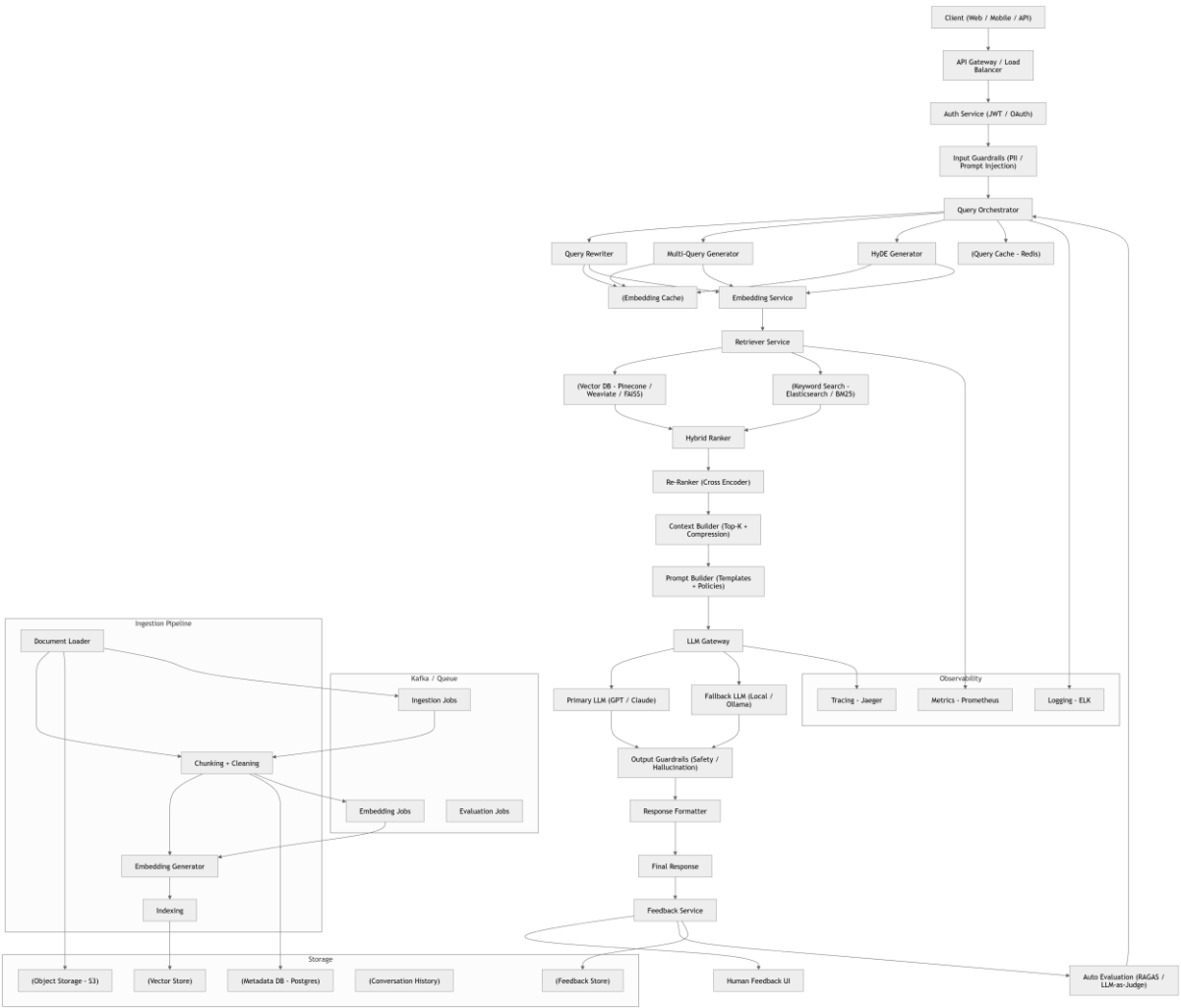


FAANG-Level Enterprise RAG Architecture



Suggested Tech Stack (Real-World)

Layer	Tools
API Gateway	Kong / NGINX
Backend	FastAPI / Node.js
Vector DB	Pinecone / Weaviate / FAISS
Search	Elasticsearch
Cache	Redis
Queue	Kafka
Storage	S3

DB	Postgres
LLM	OpenAI / Claude / Ollama
Observability	Prometheus + Grafana + ELK

◆ 1. Entry Layer

- API Gateway handles routing + rate limiting
 - Auth ensures secure access
 - Input guardrails block malicious prompts
-

◆ 2. Smart Query Processing

- Query rewriting
 - Multi-query expansion
 - HyDE (improves semantic recall)
-

◆ 3. Hybrid Retrieval (Core of RAG)

- Vector DB → semantic search
 - Elastic/BM25 → keyword match
 - Combined → hybrid ranking + re-ranking
-

◆ 4. Context Engineering (Very Important)

- Top-K selection
 - Token compression
 - Prompt templating
-

◆ 5. LLM Gateway (Production Trick)

- Routes across:
 - GPT / Claude
 - Local models (fallback)
- Enables cost + reliability control

◆ 6. Guardrails

- Input: injection, PII
 - Output: hallucination, toxicity
-

◆ 7. Async + Ingestion

- Kafka pipelines handle:
 - Document ingestion
 - Embeddings
 - Indexing
-

◆ 8. Observability

- Logs, metrics, tracing
 - 👉 Required for debugging hallucinations + latency
-

◆ 9. Feedback Loop (FAANG Secret Sauce)

- Human feedback
 - Automated evaluation (RAGAS)
 - Continuous improvement
-

⚡ What Makes This “FAANG-Level”

- ✓ Microservices
- ✓ Scalable (Kafka + caching)
- ✓ Hybrid retrieval
- ✓ Multi-model LLM routing
- ✓ Full observability
- ✓ Continuous evaluation loop

1. 🔥 Microservice Separation

Each critical function is isolated:

- Query orchestration
- Retrieval
- Ranking
- Guardrails
- LLM gateway

👉 Enables **independent scaling + deployment**

2. ⚡ Hybrid Retrieval (Must-have)

Instead of only vector search:

- **Vector DB** → semantic understanding
- **BM25 / Elastic** → keyword precision

👉 Combined via **hybrid ranking**

3. 🧠 Smart Query Expansion

You now support:

- Query rewriting
- Multi-query generation
- HyDE

👉 Boosts retrieval recall massively

4. 🛡️ Enterprise Guardrails

Split into:

- Input validation (prompt injection, PII)
 - Output filtering (toxicity, hallucination checks)
-

5. ⚡ Caching Strategy (Cost Saver)

- Redis for query caching
- Embedding reuse
- Response caching

👉 Reduces LLM cost by **30–70%**

6. 📦 Async Pipelines (Kafka)

Used for:

- Document ingestion
- Embedding generation
- Evaluation jobs

👉 Ensures system doesn't block

7. 📊 Observability (Critical)

- Logs → ELK
- Metrics → Prometheus
- Tracing → Jaeger

👉 Debug latency + hallucinations

8. 🔁 Continuous Evaluation Loop

- Human feedback
- Automated scoring (RAGAS)
- Feeds back into system

👉 This is how FAANG systems improve over time